Smart Agriculture Management System

Object Oriented Analysis and Software Engineering (OOASE) - Project Report

Submitted by:

Sanofar Fathima I

1. Introduction

1.1 Overview

The **Smart Agriculture Management System** is a web-based platform for the farmlands of Tamil Nadu. It brings live soil and land telemetry, irrigation-dam storage information, localised agronomic advice, a fertigation (fertiliser and water) calculator, government scheme information and market prices into one place, so that a farmer can take decisions from a phone or computer instead of depending on guesswork. The platform is organised around the seven agro-climatic zones of the State and the traditional Pattam (seasonal) calendar, and it is designed to work with low-cost field devices: multi-depth soil moisture probes, optical NPK sensors, solenoid drip valves and a LoRaWAN gateway.

Because farm details, land information and sensor controls are sensitive, the system uses role-based access with two kinds of signed-in users: **farmers (customers)** and the **creator administrator**. A visitor who has not signed in can read the public pages only. The application is deployed on Vercel and can be opened from any modern browser.

The system follows a client–server architecture. The responsive web front end talks to a Node.js and Express REST API, and the data (users, farmland telemetry, reviews, enquiries) is stored in a JSON file store. The front end is hosted on Vercel and the API on Render. The project is documented using OOASE practices: requirements prepared in MS Word, seven UML diagrams, implementation in VS Code, API testing with Postman and version control with Git.

1.2 Problem Statement

Most small and marginal farmers in Tamil Nadu still manage water and fertiliser by experience and by the calendar alone. This leads to recurring problems:

- **Erratic water supply** – monsoon rainfall is uneven and Cauvery basin water is scarce, yet irrigation is started and stopped by hand.
- **Over- and under-watering** – without soil moisture data, crops such as paddy, coconut and turmeric receive too much or too little water.
- **Unscientific fertiliser use** – urea, DAP and potash are applied by habit rather than by soil nutrient levels, raising cost and damaging soil.
- **No real-time soil information** – nitrogen, phosphorus, potassium, pH and micronutrient status are known only after a laboratory test.
- **Scattered advice** – crop calendars, zone details, dam levels, market prices and subsidy schemes are spread across many offices and portals.
- **Missed subsidies** – eligible farmers often do not know about micro-irrigation, solar pump and crop-insurance schemes.
- **Manual records** – farmer details, landholdings and feedback are kept on paper and are hard to search or report.

1.3 Objectives

- Provide a live land report showing soil and field telemetry together with the storage, inflow and outflow of major irrigation dams.
- Let farmers register, sign in, maintain a profile with photo, district, landholding and primary crops, and accept the terms and conditions.
- Give crop-wise fertigation prescriptions (urea, DAP, MOP, bio-fertiliser, zinc sulphate, daily water and drip run-time) from crop, acreage, growth phase and soil texture.
- Publish an agri-tech guide covering IoT probe installation, drip rules, NPK and pH thresholds, agro-climatic zones, the Pattam calendar and the nutrient deficiency matrix.
- Present Tamil Nadu government welfare schemes and subsidies, and live market (AGMARKNET) prices for important crops.
- Collect farmer reviews and support enquiries, and show project revenue and the customer directory.
- Restrict administrative functions strictly to the creator administrator and secure the system with login, role-based access and encrypted passwords.
- Model the system with standard UML diagrams and implement it with a layered, maintainable code base.

1.4 Scope of the Project

The system covers the cycle from farmer registration to advisory: the farmer registers and signs in, views the live land report and market prices, studies the guide, runs the fertigation calculator, reads about schemes and subsidies, and submits reviews or support enquiries; the administrator reviews customers, revenue and platform telemetry.

Included: registration and login, role management, farmer profile with photo, live land report, dam storage radar, farmlands gallery, agri-tech guide, fertigation calculator, government schemes, market price ticker, project revenue, customer directory, reviews, support desk and administrator console.

Not included: fully commercial IoT hardware integration, real payment gateway for subscriptions, a native mobile application and multi-language (Tamil/English) switching (see Section 6.3).

1.5 Technologies Used

The project uses web technologies that are free, well documented and run on ordinary hardware. JavaScript is used on both client and server, which keeps the code base consistent.

Layer	Technology	Purpose
Frontend	HTML5, CSS3, JavaScript (ES6)	Responsive single-page interface with sections, tabs and pop-up dialogs
Backend	Node.js, Express.js	REST API, business logic, authentication and authorisation (runs on port 3000)
Database	JSON file store (db.json), Node.js fs module	Persistence of users, farmland telemetry, reviews and enquiries

Layer	Technology	Purpose
Simulation	Telemetry sensor simulator in server.js	Generates live soil moisture and valve readings for demonstration
Security	Token sessions, role checks, CORS, input validation	Token login, creator-only gate, origin control, safe input handling
Field layer (design)	LoRaWAN soil probes, optical NPK sensors, solenoid drip valves	Soil moisture, nutrient and irrigation actuation described by the platform
Hosting	Vercel (front end), Render (API), Git	Cloud deployment of both tiers and version control
Requirements	MS Word	Requirement analysis document
Modelling	UML (use case, class, sequence, activity, state chart, component, deployment)	Analysis and design diagrams
Testing	Postman, browser testing	Collection-based API testing and manual scenarios
Tooling	VS Code, npm, Git	Code editing, packages and version control

1.6 Organisation of the Report

Section 2 presents the requirement analysis; Section 3 the seven UML diagrams; Section 4 the project code, architecture and testing; Section 5 the screens of the website; and Section 6 the conclusion, limitations, future enhancements and references.

2. Requirement Analysis

2.1 Approach

Requirements were analysed in the object-oriented style: actors were identified first, their goals were written as use cases, the real-world things they handle were identified as classes, and the connecting rules were captured as business rules. The requirements were refined by walking through the normal workflow: a farmer registers and accepts the terms, signs in, checks the live land report and market prices, uses the fertigation calculator to plan inputs, reads the guide and schemes, and gives feedback; the administrator monitors customers, revenue and sensor settings.

2.2 User Roles

Role	Description	Main responsibilities
Creator Administrator	Platform owner with full access through a separate administrative gate.	Views and manages customers, revenue, reviews and enquiries; sets global telemetry calibrations and emergency drought overrides.
Farmer (Customer)	Registered agriculturist.	Maintains profile, views live land report, uses calculator and guide, reads schemes, writes reviews, sends enquiries, overrides valves manually on own farm.
Visitor	Unregistered person.	Reads public sections (home, farmlands gallery, about); must register or sign in to use the dashboard.

2.3 Functional Requirements

ID	Requirement	Module
FR01	Register a farmer account with name, e-mail, password, phone, date of birth, district, farm size and primary crops.	Authentication
FR02	Sign in with e-mail and password; reject invalid credentials; offer quick demo accounts for evaluation.	Authentication
FR03	Require acceptance of the Terms and Conditions during registration.	Authentication
FR04	Issue a token on login and restrict each page and API call by the user role.	Authentication
FR05	Provide a separate administrative gate that accepts the creator only and rejects farmer logins.	Administration
FR06	Allow a farmer to upload a profile picture and edit profile details.	Profile
FR07	Show the live land report with soil telemetry for agro-climatic zones.	Live report
FR08	Show dam storage radar: level, percentage full, inflow and outflow for major dams (Mettur, Bhavanisagar, Vaigai, Aliyar).	Live report
FR09	Display live market prices (AGMARKNET) with change indicators in a ticker.	Market
FR10	Display a gallery of Tamil Nadu farmlands and agro-zones.	Gallery
FR11	Provide the agri-tech guide with modules on probes, drip rules, NPK and pH thresholds and governance.	Guide
FR12	Present the seven agro-climatic zones, the Pattam calendar and the soil NPK and micronutrient matrix.	Guide

ID	Requirement	Module
FR13	Present water budgeting, drip maintenance and energy management protocols.	Guide
FR14	Present Tamil Nadu government schemes with department, subsidy percentage and benefits.	Schemes
FR15	Generate a fertigation prescription from crop, acreage, growth phase and soil texture.	Calculator
FR16	Show project revenue, revenue distribution by stream and subscription plans (Uzhavan Starter, Kaveri Precision Pro, Agro-Estate Enterprise).	Revenue
FR17	Show the customer directory with farmer details and landholdings.	Customers
FR18	Let a farmer submit a review with rating, headline, feedback and avatar photo.	Reviews
FR19	Let any user send a support or sensor-installation enquiry to the engineering desk.	Support
FR20	Hold scheduled irrigation when heavy rain is forecast and shut valves when rainfall exceeds the limit.	Irrigation rules
FR21	Give the administrator a command centre showing platform telemetry.	Administration
FR22	Keep customer data and administrative actions inaccessible to farmers.	Security

2.4 Non-Functional Requirements

Attribute	Requirement
Security	Passwords stored only as hashes; token required for protected API calls; administrator gate restricted to the creator; secrets kept in environment variables; land coordinates and survey numbers protected.
Performance	Pages and sections load within about two seconds on a normal connection; images are compressed before upload for fast loading.
Usability	Clear section menu, high-contrast dark theme, quick demo logins, icons, and short guided modules suitable for farmers.
Reliability	Input is validated on client and server; safe defaults hold irrigation when rainfall is forecast; farmers can always override valves manually.
Maintainability	Server code separated into models, routes, middleware and services; client code into sections, scripts and styles.
Portability	Stateless REST API; runs in any modern browser; responsive layout for phone and desktop.
Availability	Hosted on Vercel and Render; target probe-grid uptime shown to users is 99.98%.

2.5 Hardware and Software Requirements

Item	Minimum requirement
Processor, memory, storage	Dual-core processor, 4 GB RAM (8 GB recommended for development), 2 GB free disk space
Operating system / runtime	Windows 10 or later, Ubuntu 20.04 or later, or macOS; Node.js 18 or newer and npm
Database / browser	No separate database server (JSON file store); current Chrome, Edge, Firefox or Safari
Field devices (design)	LoRaWAN soil moisture probe, optical NPK sensor, solar battery node, solenoid valve and district gateway
Tools	VS Code, Postman, MS Word, UML tool, Git

2.6 Data Requirements

Entity	Principal attributes	Description
User	userId, name, email, passwordHash, phone, dob, role, isActive	A person who can log in; the role decides what they may do.
Farmer profile	district, farmSizeAcres, primaryCrops, photo, termsAccepted	Agricultural details of a customer.
Farm	farmId, district, zone, soilType, areaAcres	A land holding owned by a farmer.
SensorReading	readingId, moisture, pH, N, P, K, temperature, takenAt	One telemetry record from a probe.
Dam	damId, name, basin, level, capacity, inflow, outflow	Irrigation dam storage record.
MarketPrice	crop, market, price, unit, changePercent	AGMARKNET price entry.
Prescription	crop, acres, phase, soilTexture, ureaKg, dapKg, mopKg, waterLitres	Output of the fertigation calculator.
Review	name, district, crops, rating, headline, feedback, avatar	Farmer testimonial.
Inquiry	name, email, phone, district, message, createdAt	Support or installation request.

2.7 Business Rules

- Only the creator administrator may use the administrative gate; farmer logins are rejected there and customers have no administrative privileges.
- A farmer must accept the Terms and Conditions before registration is complete.
- Farmers retain manual physical override rights on their farm valves at all times.
- An emergency shut-off is triggered automatically if rainfall exceeds 15 mm; scheduled cycles hold if more than 10 mm is forecast within 12 hours.
- Drip actuation follows soil texture: sandy loam starts below 55% moisture and stops at 82%; coconut rings start below 50% field capacity; acidic hill soil suppresses irrigation in morning mist or humidity above 85%.
- Fertigation quantities are calculated per acre of the selected crop and growth phase and scaled by the land area entered.
- Land coordinates and survey numbers are encrypted and not shared with unauthorised third parties.
- A deactivated user can no longer log in, but historical reviews and records are kept.

2.8 Feasibility Study

- **Technical:** all software is open source or free-tier, well documented and needs only JavaScript skills and ordinary hardware.
- **Economic:** there is no licence cost; Vercel and Render free tiers are enough, and the JSON file store needs no database licence.
- **Operational:** quick demo accounts and a one-click creator verification make evaluation easy for examiners.

2.9 Assumptions and Constraints

- A reliable network connection to the server, and each user has an individual account.
- Field telemetry shown in the demonstration is sample data; real sensors are connected through the gateway in a full deployment.
- Market prices, dam levels, revenue and subscription figures are shown as demonstration data and are not official figures in this version.
- Demo credentials are for testing only and must never be used in a deployed environment.

3. UML Diagrams

3.1 Overview

The Unified Modeling Language (UML) is a standard visual language for specifying and documenting software. The seven diagrams below describe the system from the viewpoint of its users (use case), its structure (class, component, deployment) and its behaviour (sequence, activity, state chart).

3.2 Diagrams and Descriptions

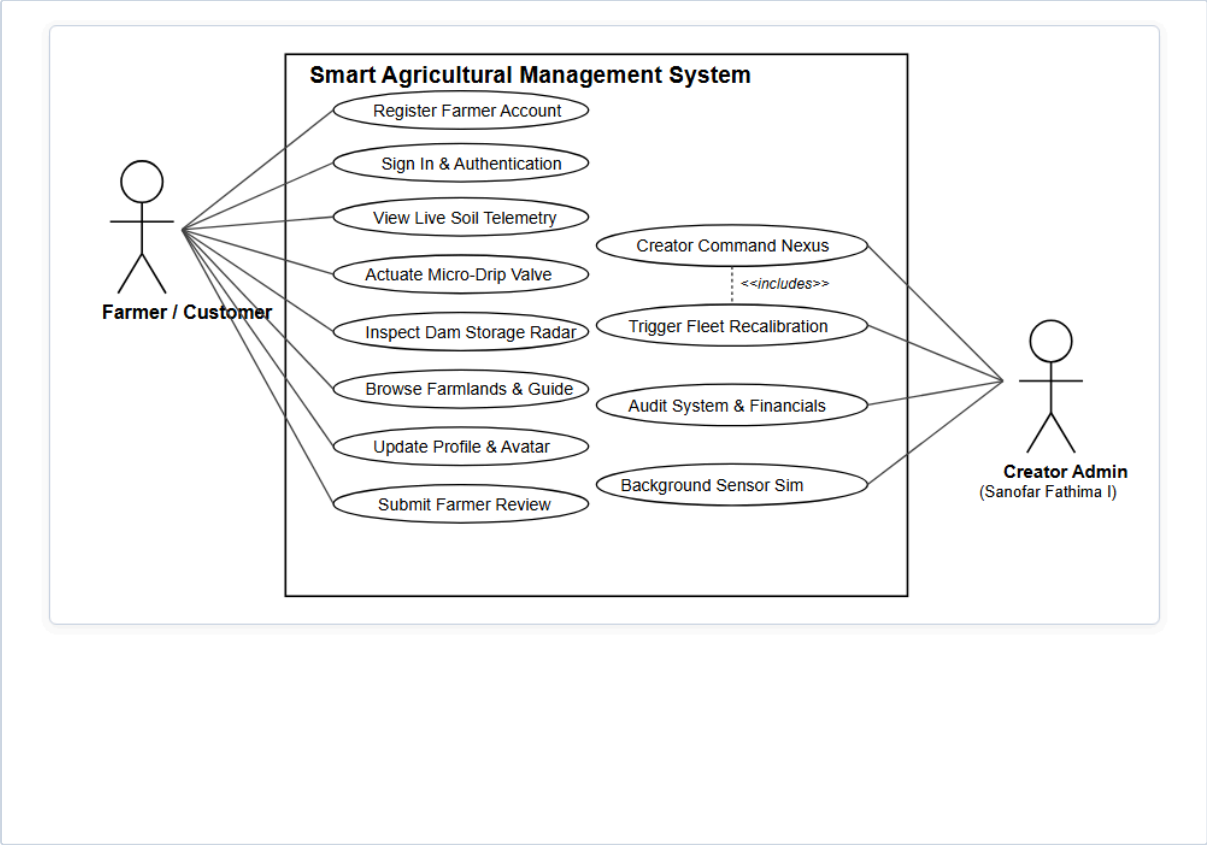


Figure 3.1: Use Case Diagram

Use case diagram. Two main actors use the system. The Farmer registers or signs in, views the live land report, reads the guide, uses the fertigation calculator, checks schemes and market prices, edits the profile, writes reviews and sends enquiries. The Creator Administrator signs in through the administrative gate, views customers and revenue, moderates reviews, answers enquiries and manages platform telemetry. The IoT gateway acts as a secondary actor that supplies sensor readings. A visitor who has not signed in can read public sections only.

3.3 Class Diagram

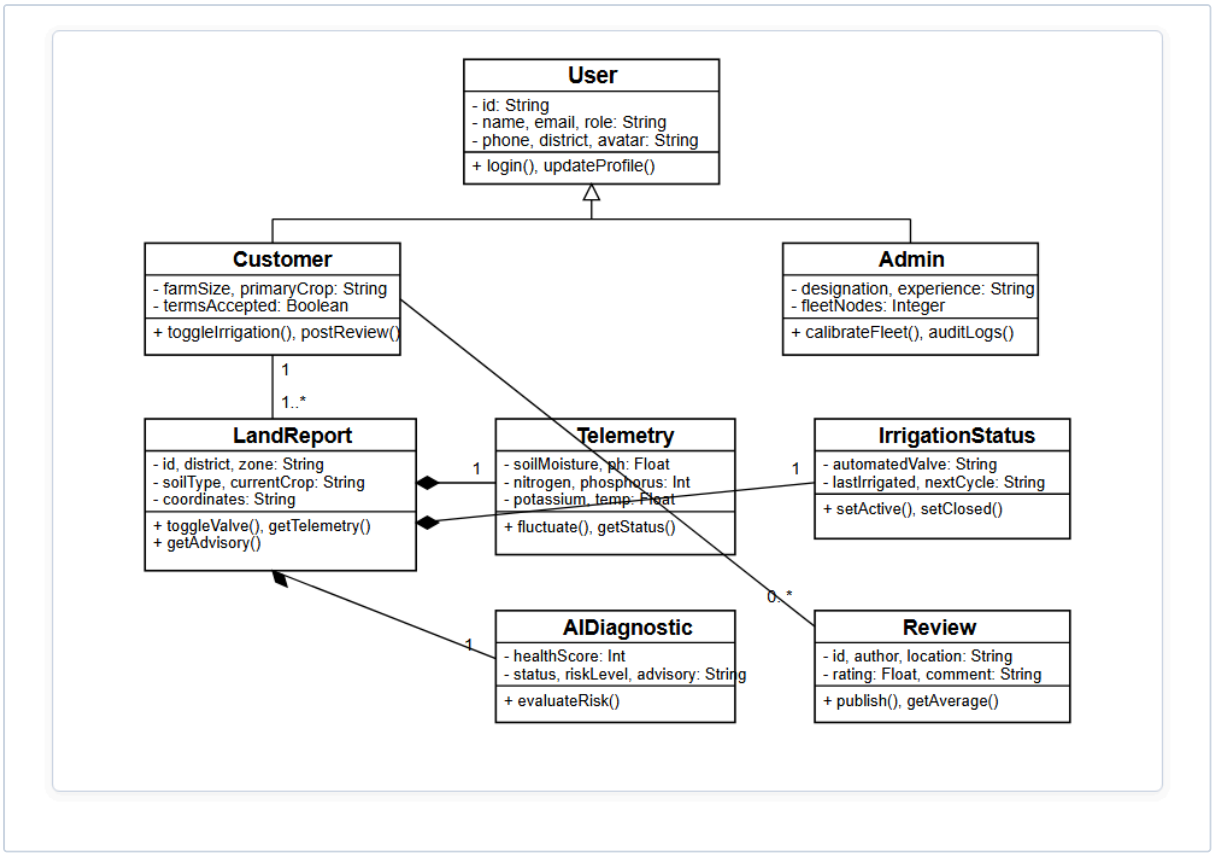


Figure 3.2: Class Diagram

Class diagram. User is the parent class; Admin and Farmer inherit from it. A Farmer owns one or more Farms; each Farm has SensorNodes that produce SensorReadings and IrrigationValves that are actuated by rules. A Farmer requests Prescriptions, writes Reviews and may activate a SubscriptionPlan. Dams, market prices and enquiries are handled as simple records of the live report, ticker and support desk. Operations include register(), login(), calculatePrescription(), openValve(), closeValve() and submitReview().

3.4 Sequence Diagram

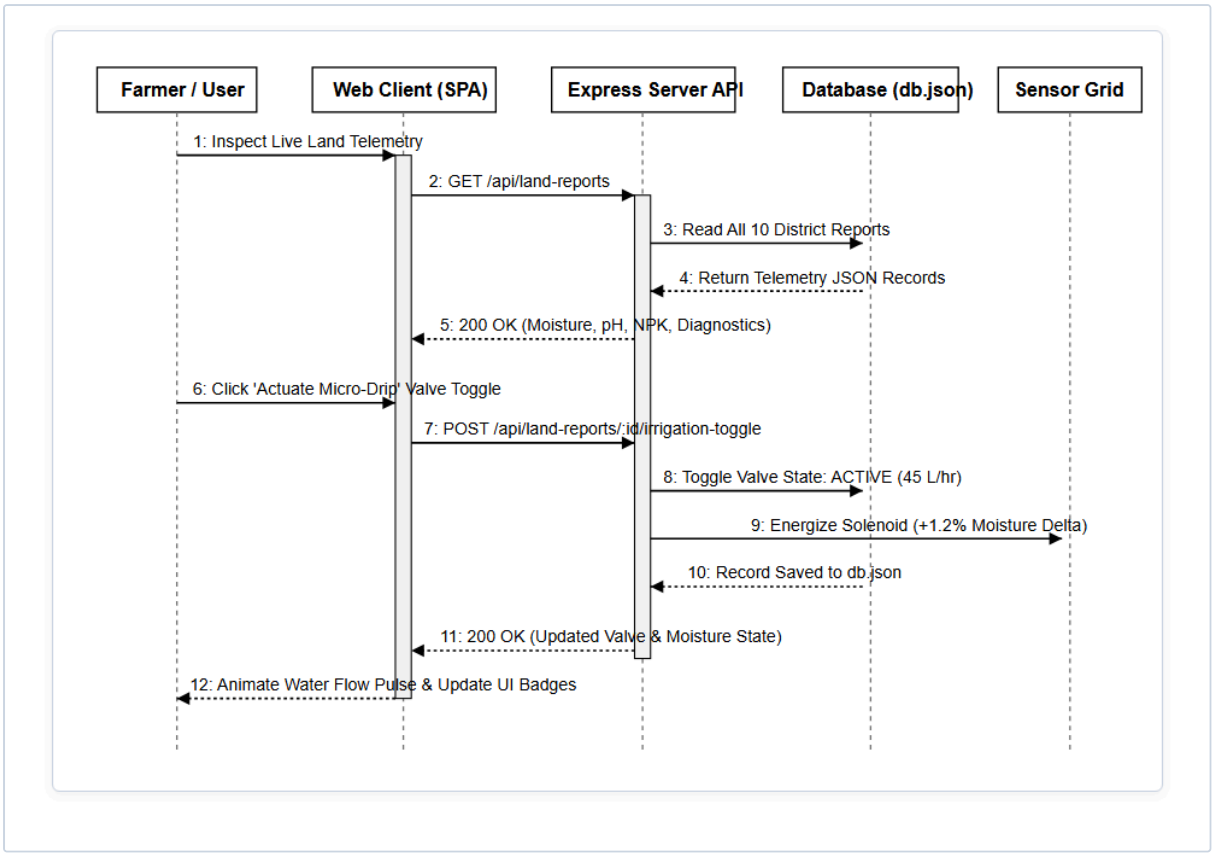


Figure 3.3: Sequence Diagram – Generate Fertigation Prescription

Sequence diagram. The farmer selects crop, area, growth phase and soil texture in the web app and presses the calculate button. The app calls the API with the token; the server verifies the token and role, loads the per-acre norms for the crop and phase, computes urea, DAP, MOP, bio-fertiliser, zinc sulphate, daily water and drip run-time, stores the prescription, and returns the result, which is displayed to the farmer. If a value is missing or invalid, an error is returned and nothing is stored.

3.5 Activity Diagram

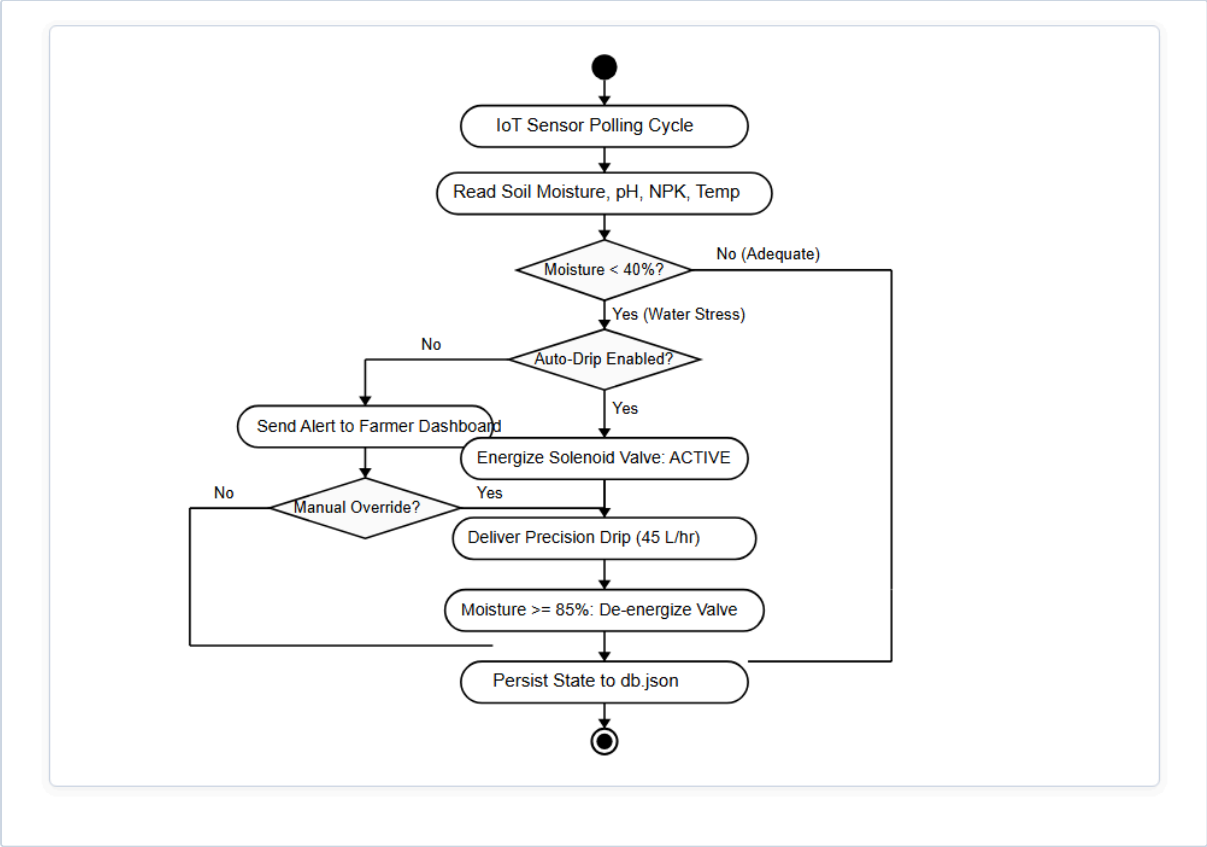


Figure 3.4: Activity Diagram – Register, Monitor and Irrigate

Activity diagram. The farmer registers, accepts the terms and signs in. The dashboard shows the live land report. If soil moisture is below the crop threshold and no heavy rain is forecast, the drip valve is opened and closed again at the upper limit; if rainfall exceeds the limit, the valve is shut and irrigation is held. The farmer may then run the calculator, read the guide and submit a review.

3.6 State Chart Diagram

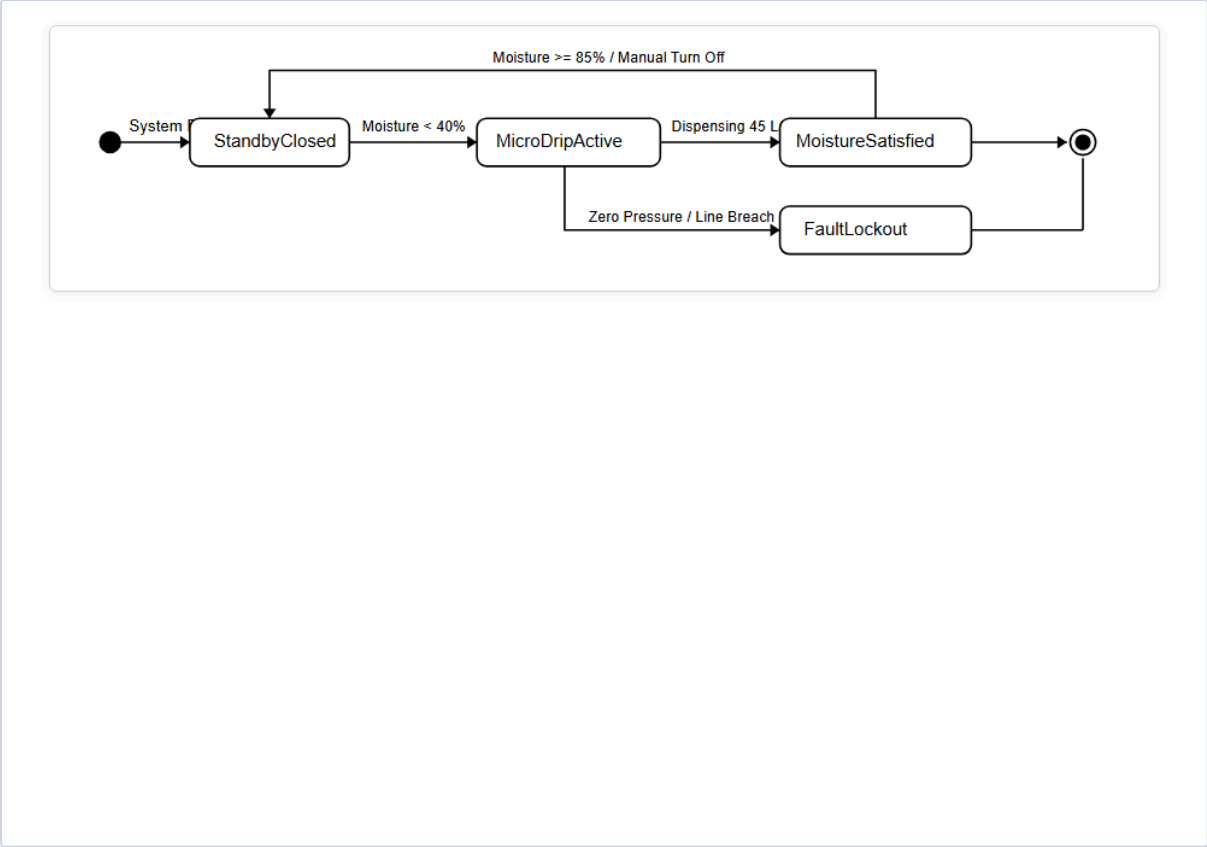


Figure 3.5: State Chart Diagram – Irrigation Valve and Farmer Account

State chart diagram. An irrigation valve starts Closed, becomes Open when soil moisture falls below the crop threshold, returns to Closed at the upper limit, moves to Held when heavy rain is forecast, and to Emergency Shut-off when rainfall exceeds the limit; a farmer may force Manual Override at any time. A farmer account is Registered after sign-up, Active after the terms are accepted and login succeeds, and Deactivated if the administrator disables it.

3.7 Component Diagram

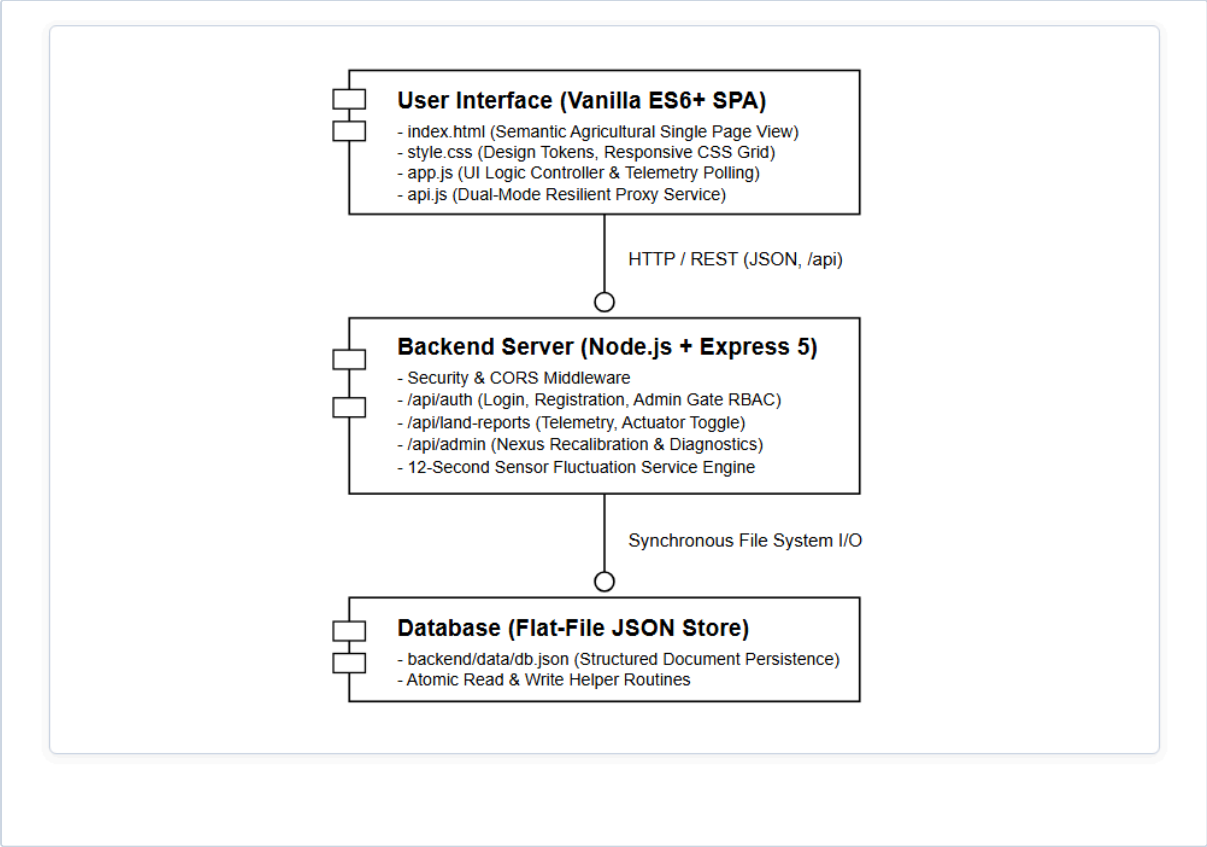


Figure 3.6: Component Diagram

Component diagram. The web user interface (sections, dialogs, calculator script) talks to the Node.js and Express backend over HTTP/REST using JSON. The backend contains security middleware, authentication, route modules and the fertigation and irrigation-rule services, and reads and writes the JSON data store (db.json). A built-in sensor simulator produces telemetry in this version; the planned IoT gateway would post real sensor readings to the backend.

3.8 Deployment Diagram

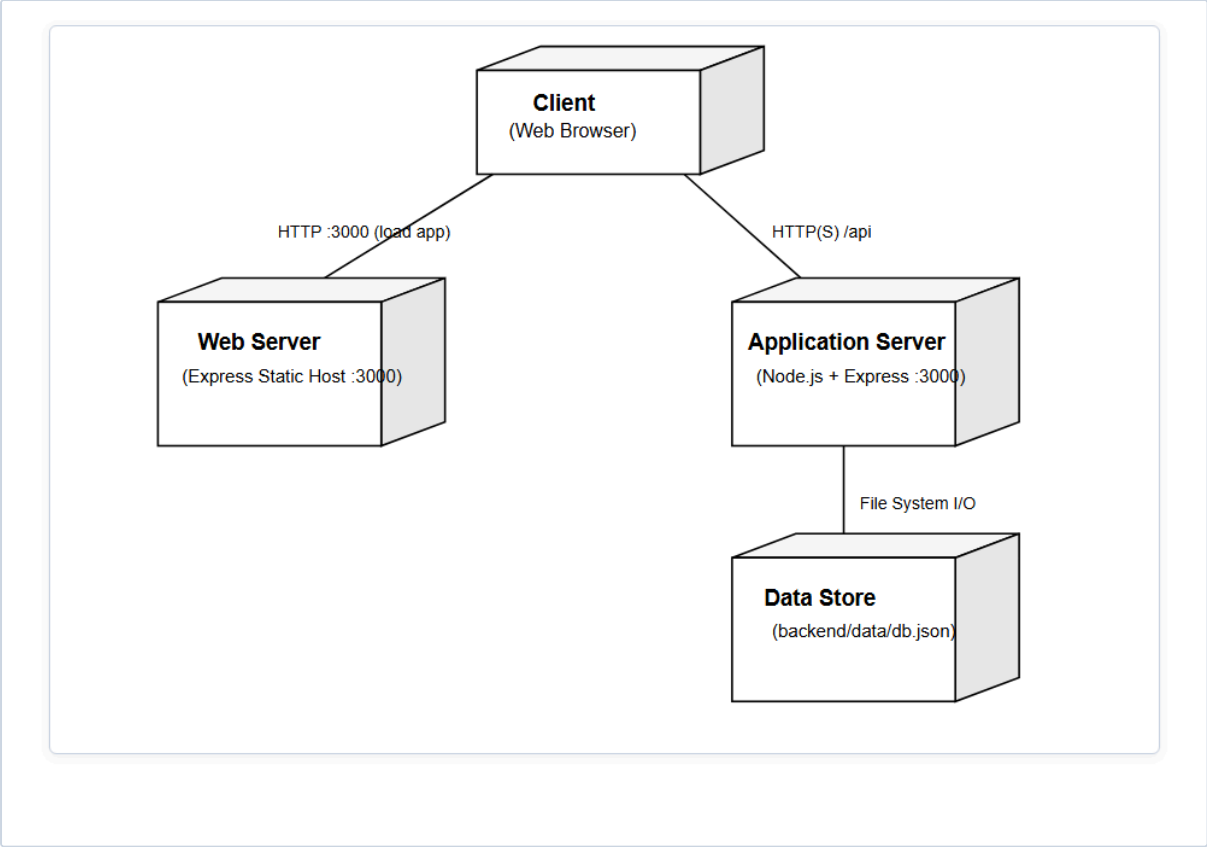


Figure 3.7: Deployment Diagram

Deployment diagram. The browser loads the application from the Vercel-hosted web server and calls the application server (Express, port 3000, path /api). Vercel rewrites /api requests to the API hosted on Render, which reads and writes the db.json data store. The field devices and LoRaWAN gateway are the planned hardware layer. In development all nodes run on localhost; in production HTTPS is used.

4. Project Coding

4.1 Architecture Overview

The system is a three-tier application. The presentation tier is the browser interface built with HTML, CSS and JavaScript; the application tier is a Node.js and Express REST API that validates requests and applies the business rules; the data tier is the JSON file store (backend/data/db.json) accessed through the Node.js file system module. Client and server communicate only through JSON over HTTP.

A request flows as follows: the page calls the API and attaches the login token; Express applies the security middleware and routes the request to the matching route module; the authenticate middleware checks the token and role; the handler reads or writes db.json; and the JSON result is rendered in the page.

4.2 Project Directory Structure

```
agriculture/
|-- backend/                     # Node.js & Express REST API server
|   |-- data/
|   |   `-- db.json              # JSON persistence store (users, telemetry)
|   |-- server.js                # Express API server & telemetry simulator
|   |-- package.json             # Backend dependencies
|   `-- package-lock.json
|-- frontend/                    # Responsive web client (single page app)
|   |-- assets/
|   |   `-- images/              # Agro-climatic terrain imagery
|   |-- css/
|   |   `-- style.css            # Obsidian black, white & cobalt blue theme
|   |-- js/
|   |   |-- api.js               # API client with offline seed failover
|   |   `-- app.js               # UI event controllers, dashboard views
|   |-- index.html               # Single page application entry point
|   `-- vercel.json              # Frontend rewrite configuration (Vercel)
|-- .gitignore                   # Ignore rules (node_modules, system files)
|-- package.json                 # Root configuration and dev scripts
|-- package-lock.json
|-- vercel.json                  # Root Vercel routes & Render API proxy
|-- OOASE_Project_Report_Smart_Agriculture.pdf # Academic project report
`-- README.md                    # Project documentation
```

4.3 Environment Configuration

Deployment-specific values are read from environment variables. The template .env.example is committed, each developer copies it to .env, and the real file is never committed, so the allowed origin and creator e-mail stay out of version control.

```
PORT=3000
NODE_ENV=development
CLIENT_ORIGIN=https://smart-agri-frontend-rho.vercel.app
CREATOR_EMAIL=<creator e-mail>
DB_FILE=./data/db.json
SIM_INTERVAL_MS=30000
```

4.4 Server Entry Point (server.js)

backend/server.js is the single entry point. It creates the Express application, enables CORS and JSON parsing, defines the JSON store helpers and the token and role middleware, starts the telemetry sensor simulator, mounts the API routes and listens on the configured port.

```
// ---- Routes ----
app.get('/api/health', (req, res) => res.json({ status: 'ok' }));
app.post('/api/auth/register', registerFarmer); // terms must be accepted
app.post('/api/auth/login', loginFarmer);
app.post('/api/auth/admin', loginCreator); // creator-only gate
app.get('/api/live/dams', (req, res) => res.json(readDb().dams));
app.get('/api/live/farmlands', (req, res) => res.json(readDb().farmlands));
app.post('/api/prescriptions', authenticate, createPrescription);
app.get('/api/reviews', (req, res) => res.json(readDb().reviews));
app.post('/api/reviews', authenticate, addReview);
app.post('/api/inquiries', addInquiry);
app.get('/api/admin/customers', authenticate, authorize('admin'), listCustomers);
app.get('/api/admin/revenue', authenticate, authorize('admin'), getRevenue);

app.listen(PORT, () => console.log('Smart Agri API running on port ' + PORT));
```

Public routes (health, register, login, live report, reviews list and enquiries) skip authentication; prescription and review-posting routes call authenticate; every /api/admin route also calls authorize(admin), which accepts only the creator account. The simulator updates soil moisture and valve state of each farmland at a fixed interval using the irrigation rule engine of Section 4.6.

4.5 Data Store and Collections

All data is held in backend/data/db.json as JSON collections (arrays of objects) that are read and written by server.js; each collection implements a class of the class diagram.

Collection	Contents	Class diagram entity
users	Name, e-mail, hashed password, phone, date of birth, role, active status, photo.	User, Admin, Farmer
farmlands	District, zone, soil type, area, owner, latest moisture, pH, NPK, temperature and valve state.	Farm
readings	Moisture, pH, N, P, K, temperature and time stamp.	SensorReading
dams	Name, basin, level, capacity, inflow and outflow.	Dam
prescriptions	Crop, acres, phase, soil texture and computed quantities.	Prescription
reviews	Rating, headline, feedback, crops and avatar of a farmer.	Review
inquiries	Name, contact details, district and message.	Inquiry

4.6 REST API Routes

The API is served under /api (http://localhost:3000/api).

Method	Endpoint	Description	Access
GET	/health	API status.	Public
POST	/auth/register	Register a farmer account (201).	Public
POST	/auth/login	Authenticate and receive a token.	Public
POST	/auth/admin	Creator administrative sign-in.	Creator
GET	/live/dams	Dam storage radar data.	Public
GET	/live/farmlands	Farmland telemetry from the simulator.	Public
GET	/live/prices	Market price ticker.	Public
POST	/prescriptions	Calculate and store a fertigation prescription.	Farmer
GET / PUT	/farmers/me	View or update own profile.	Farmer

Method	Endpoint	Description	Access
GET / POST	/reviews	List or submit reviews.	Public / Farmer
POST	/inquiries	Send an enquiry to the support desk.	Public
GET	/admin/customers	Customer directory.	Creator
GET	/admin/revenue	Revenue and commercial metrics.	Creator

The key business logic is the fertigation service. Per-acre norms for the chosen crop and phase are multiplied by the land area; for example, for 5 acres of Samba/Kuruvai paddy in the vegetative stage the result is 275 kg urea, 200 kg DAP, 125 kg MOP, 20 bio-fertiliser packets and 57,500 litres of water per day with a 90-minute drip cycle.

```
function prescribe(crop, acres, phase, soil) {
  const n = NORMS[crop][phase]; // per-acre norms
  return {
    ureaKg: n.urea * acres,
    dapKg: n.dap * acres,
    mopKg: n.mop * acres,
    bioPackets: n.bio * acres,
    zincKgPerAcre: n.zinc,
    waterLitresPerDay: n.water[soil] * acres,
    dripMinutes: n.runtime
  };
}
```

The irrigation rule engine decides valve actions from soil moisture and rainfall:

```
function decide(reading, rule, rain) {
  if (rain.mm > 15) return 'EMERGENCY_SHUTOFF';
  if (rain.forecastMm12h > 10) return 'HOLD';
  if (reading.moisture < rule.startBelow) return 'OPEN';
  if (reading.moisture >= rule.stopAt) return 'CLOSE';
  return 'NO_CHANGE';
}
```

4.7 Frontend Implementation

The front end is a responsive single-page website organised around a few reusable pieces:

- A left sidebar menu lists the sections: Live Land Report, Farmlands Gallery, Agri-Tech Guide, Project Revenue, Customer Directory, Farmer Reviews, Creator and About & Support.
- A sign-in dialog with three tabs (Sign In, Sign Up as Farmer, Creator Admin) validates inputs, shows quick demo accounts and the administrative gate.
- A profile dialog lets a farmer edit details and upload a photo, which is compressed before it is sent.
- The guide uses tabs (core modules, zones, Pattam calendar, NPK matrix, water budgeting, schemes, calculator) and tables for quick reading.
- A calculator form collects crop, area, phase and soil and displays the prescription instantly.
- The market ticker, dam radar and statistic counters are reusable components fed by the API.
- api.js is an API client that falls back to built-in seed data when the server is unreachable; app.js holds the event controllers and dashboard views; style.css defines the obsidian-black, white and cobalt-blue theme.

4.8 API Testing with Postman

Every endpoint group was tested with Postman. The collection defines the variable baseUrl (http://localhost:3000/api) together with the tokens needed to chain requests (farmerToken, adminToken). Each request carries test scripts (for example, health returns 200 and register returns 201), so the whole collection can run in sequence after resetting db.json.

4.9 Test Scenarios

Scenario	Expected result
Sign in with correct e-mail and password	The farmer is authenticated and a token is returned.
Sign in with a wrong password	The request is rejected and no token is issued.
Register without accepting the terms	Registration is blocked with a validation message.
Farmer signs in at the creator gate	Access is denied and the farmer is returned to the dashboard.
Open a protected route without a token	The request is rejected as unauthorised.
Calculate prescription for 5 acres paddy (vegetative)	Urea 275 kg, DAP 200 kg, MOP 125 kg, 20 packets, 57,500 L/day, 90 minutes.
Submit calculator with empty area	Validation error; nothing is stored.
Rainfall above 15 mm	Valve action is Emergency Shut-off.
Forecast above 10 mm in 12 hours	Scheduled irrigation is held.
Upload a large profile photo	The photo is compressed and saved.
Submit a review and an enquiry	Both are saved and the review appears in the live feed.

4.10 Deployment and Version Control

The source code is managed with Git. The front end is deployed to Vercel, which builds and publishes the site on every push and provides the public address https://smart-agri-frontend-rho.vercel.app. The root vercel.json and frontend/vercel.json define the routes and proxy /api requests to the Node.js API hosted on Render, whose allowed origin is set through the CLIENT_ORIGIN variable. The root package.json provides scripts to run the backend and frontend together during development. Environment values are configured in the hosting dashboards and never stored in the repository.

5. Website Screenshots

5.1 Overview

The screens below show the Smart Agriculture Management System running at https://smart-agri-frontend-rho.vercel.app. The signed-in screens share one layout with a left sidebar menu, a session bar showing the logged-in user and a dark, high-contrast theme.

5.2 Application Screens

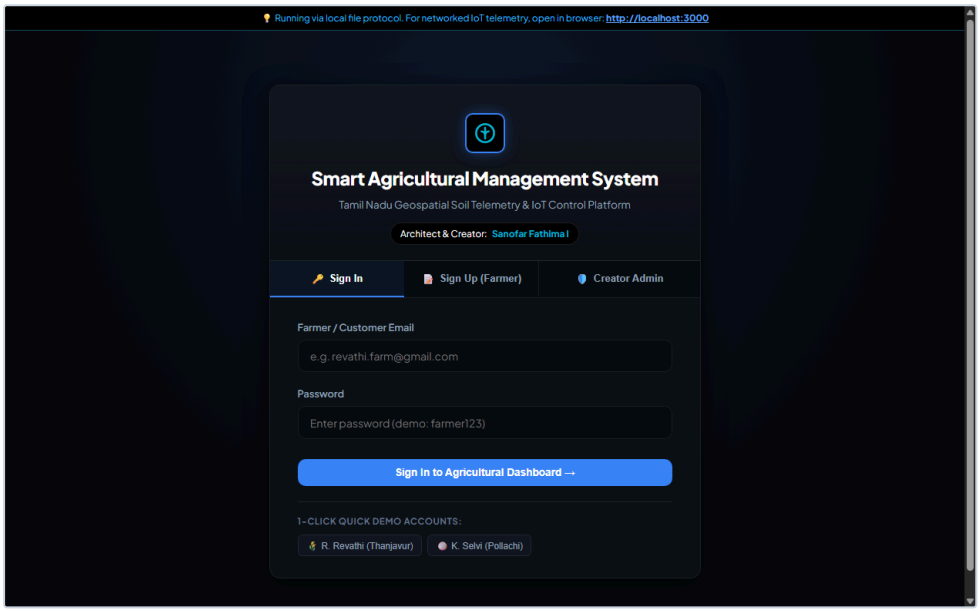


Figure 5.1: Sign In page

Sign In page. Farmers sign in with e-mail and password. Quick demo accounts (R. Revathi of Thanjavur and K. Selvi of Pollachi) fill in credentials for evaluation.

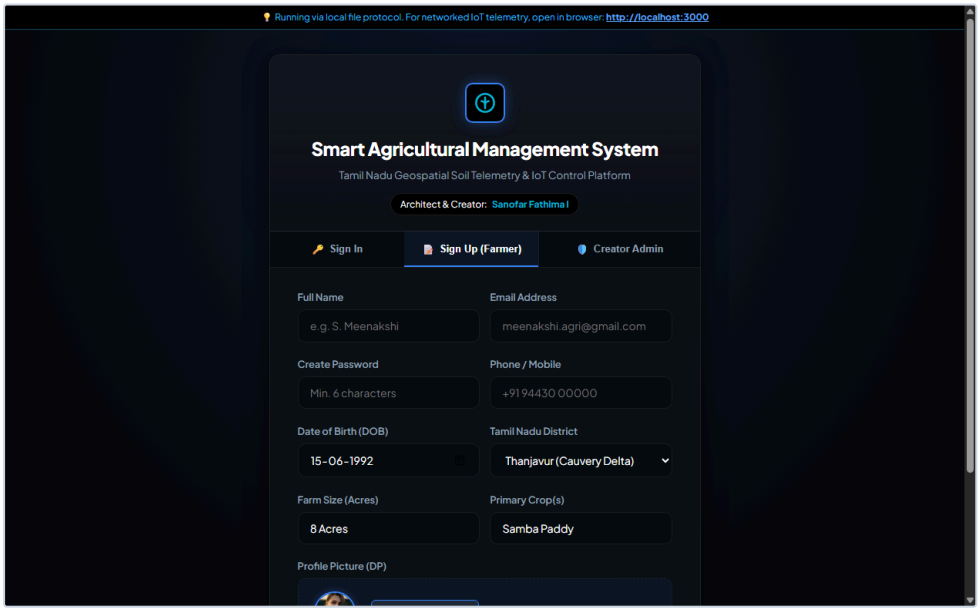


Figure 5.2: Farmer Sign Up

Sign Up page. A new farmer enters name, e-mail, password, phone, date of birth, district, farm size, primary crop and profile photo, and must accept the Terms and Conditions.

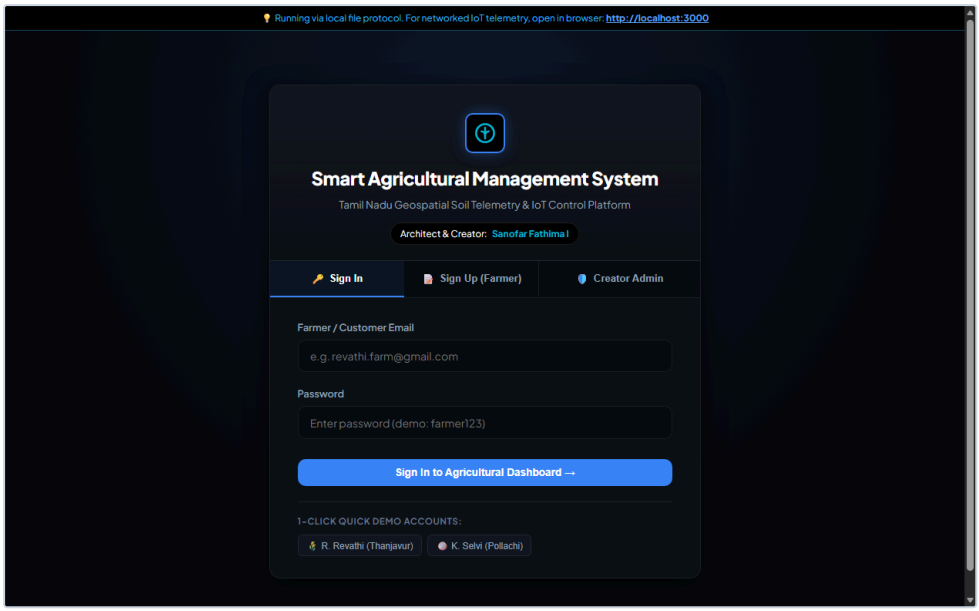


Figure 5.3: Home page and market ticker

Home page. The hero section shows the platform title, key figures (acres monitored, water conserved, probe-grid uptime) and the AGMARKNET price ticker, with the sidebar menu and signed-in farmer session.

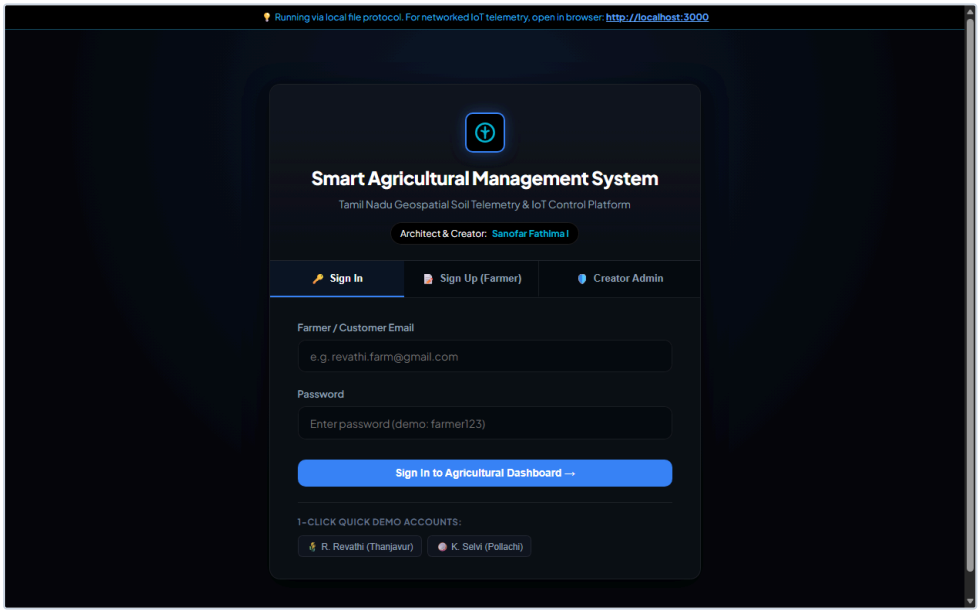


Figure 5.4: Live Land Report and dam radar

Live Land Report. The dam storage radar shows Mettur, Bhavanisagar, Vaigai and Aliyar with level, percentage full, inflow and outflow; zone tabs show soil moisture, pH, NPK and temperature.

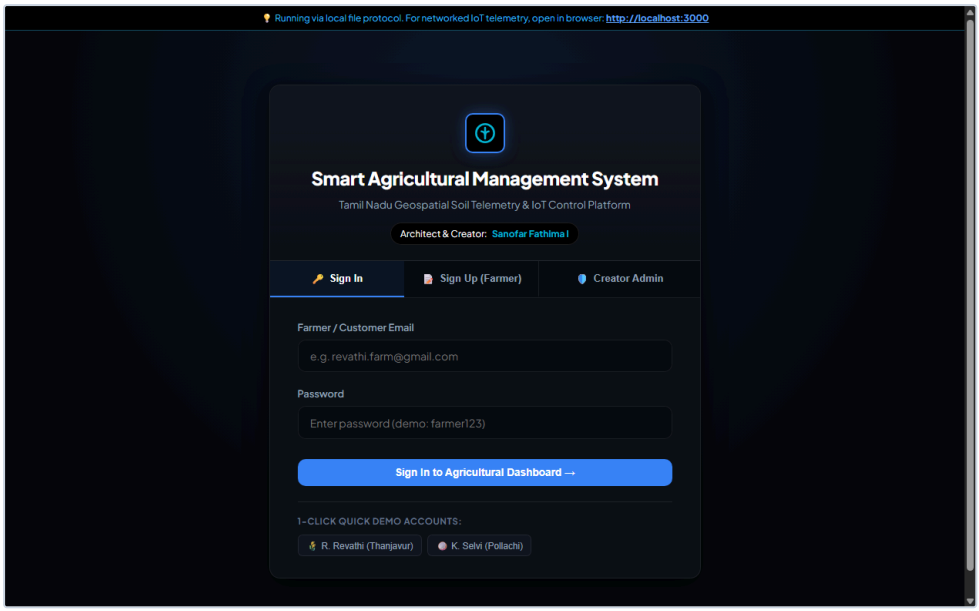


Figure 5.5: Farmlands gallery

Farmlands gallery. Cards for Thanjavur, Pollachi, Erode, the Nilgiris, Madurai and Tirunelveli show location, primary crop, soil type and moisture with an Inspect Telemetry button.

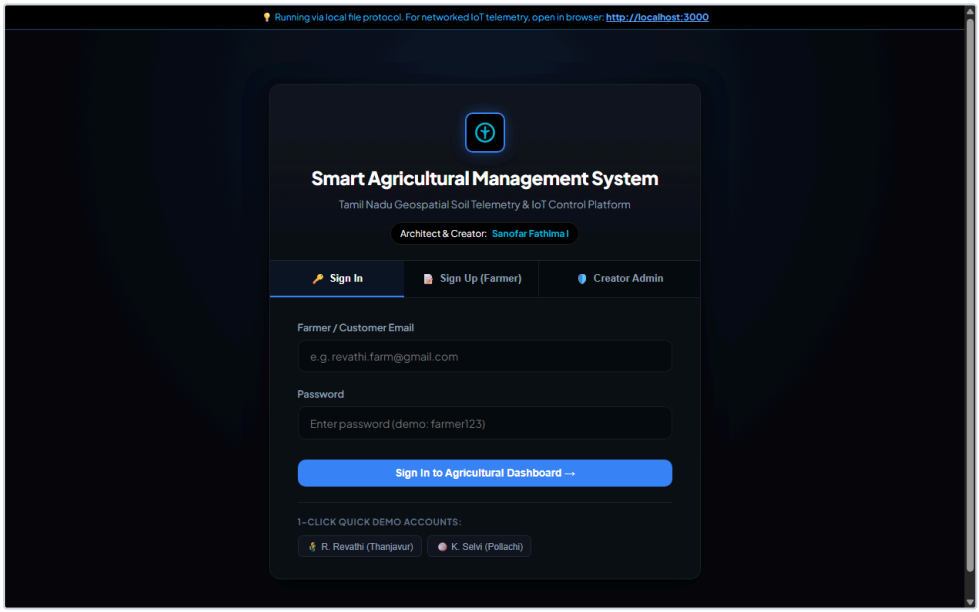


Figure 5.6: Agri-Tech Guide

Agri-Tech Guide. Tabs present core modules, agro-climatic zones, the Pattam calendar, the NPK deficiency matrix, water budgeting and government schemes.

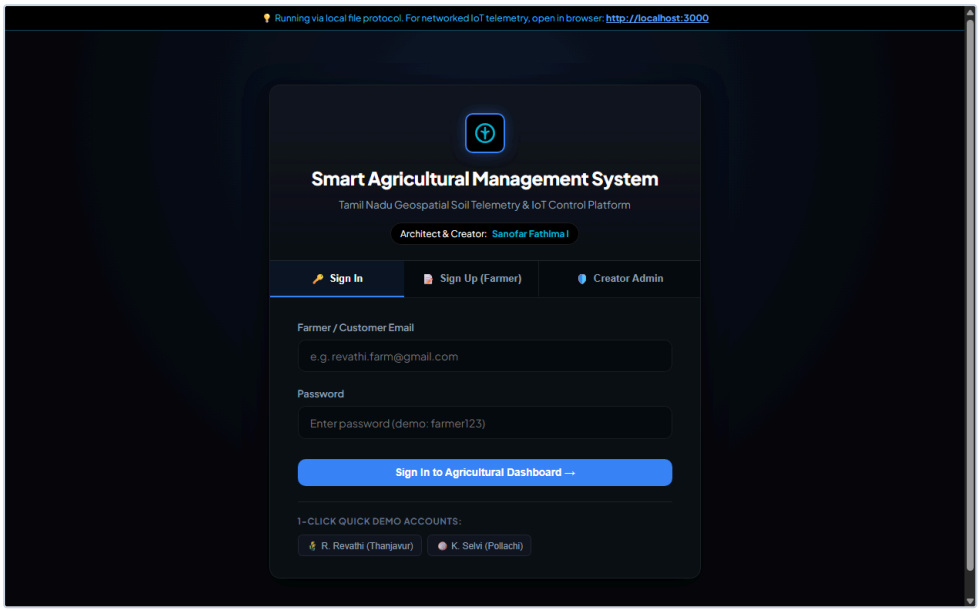


Figure 5.7: Project Revenue (creator view)

Project revenue. The creator sees total and monthly recurring revenue, farmland monitored, water conserved and the revenue distribution by agri-tech stream.

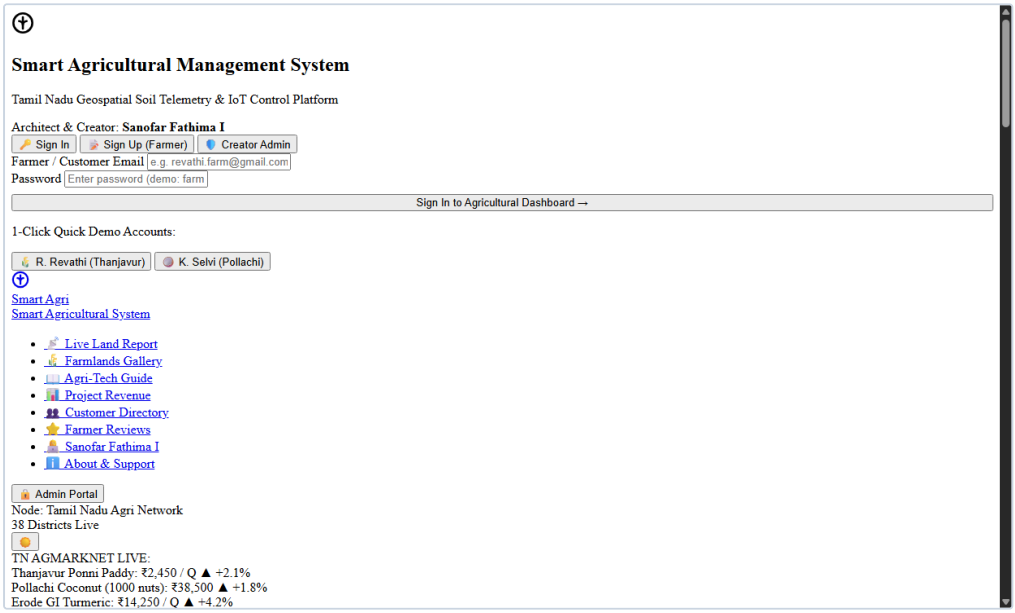


Figure 5.8: Subscription plans

Subscription plans. Uzhavan Starter, Kaveri Precision Pro (most popular) and Agro-Estate Enterprise are listed with prices and features; the Customer Details section follows.

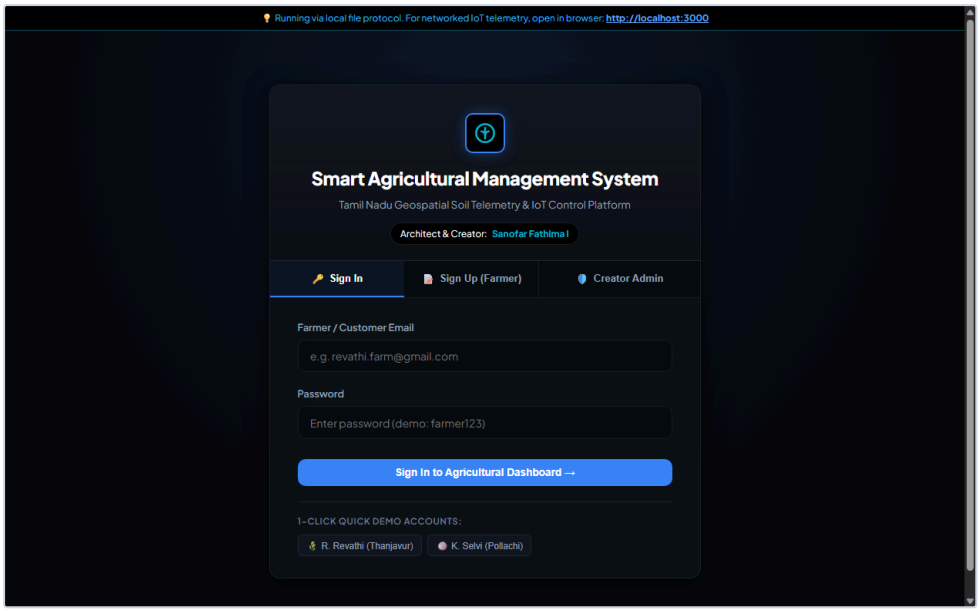


Figure 5.9: Customer Reviews and Field Feedback

Customer reviews. Verified farmers from the Cauvery Delta, Pollachi, Erode, the Nilgiris, Madurai and Tirunelveli share ratings and feedback.

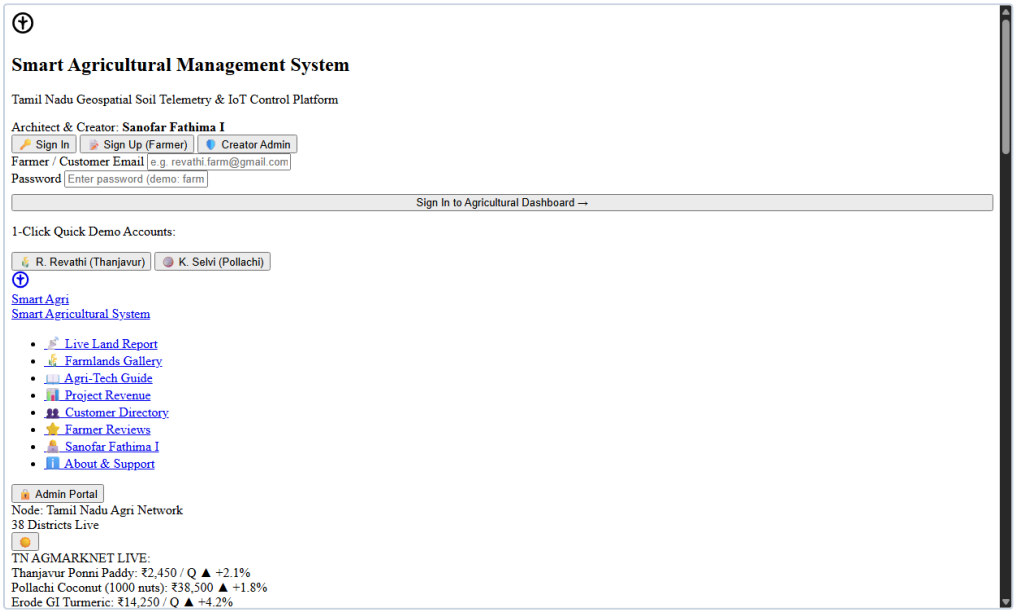


Figure 5.10: Project Creator and Support desk

Creator and support. The creator section describes the architecture and vision of the platform, and the Agricultural Support Desk form sends enquiries to the engineering team.

6. Conclusion and References

6.1 Conclusion

The Smart Agriculture Management System shows how object-oriented analysis and modern web technology can support farmers of Tamil Nadu. The requirement analysis identified the user roles and a set of functional and non-functional requirements; the UML diagrams translated them into a clear design (use case for functions, class for data, sequence, activity and state chart for behaviour, component and deployment for architecture); and the design was implemented as a responsive web front end, a Node.js and Express REST API and a JSON file data store.

The finished system lets farmers view live land and dam information, plan fertiliser and water with the fertigation calculator, learn from the guide, discover subsidies and share feedback, while the creator administrator controls the platform through a restricted gate. Security rests on hashed passwords, token authentication, role-based access and origin control, and the work was verified with Postman and test scenarios. The project also gave practical experience of the complete software engineering cycle.

6.2 Limitations

- Sensor data come from a built-in simulator, and dam levels and market prices are demonstration values; there is no live integration with official sources or hardware in this version.
- Data are kept in a JSON file store, which suits a demonstration but not heavy concurrent use.
- No online payment for subscriptions.
- The interface is a responsive web application; there is no native mobile app.
- Notifications are shown in the application only; SMS alerts for motor trips and dry runs are designed but not implemented.
- The interface language is English only.

6.3 Future Enhancements

- Live integration with LoRaWAN sensors, solenoid valves and satellite NDVI crop-health maps in place of the simulator.
- Migration of the JSON store to a database such as MongoDB or PostgreSQL.
- Official data feeds for dams (PWD) and market prices (AGMARKNET).
- Tamil and English language switching and voice support for farmers.
- SMS and WhatsApp alerts for irrigation, rainfall and pump faults.
- Mobile application with offline support for fields with weak networks.
- Machine-learning yield prediction and pest-disease detection from photographs.

6.4 References

1. Booch, G., Rumbaugh, J. and Jacobson, I., The Unified Modeling Language User Guide, Addison-Wesley.

2. Sommerville, I., Software Engineering, Pearson.

3. Pressman, R. S., Software Engineering: A Practitioner's Approach, McGraw-Hill.

4. Tamil Nadu Agricultural University (TNAU) Agritech Portal. https://agritech.tnau.ac.in

5. Tamil Nadu Agriculture Department. https://www.tn.gov.in

6. Agmarknet – Agricultural Marketing Information Network. https://agmarknet.gov.in

7. Node.js Documentation. https://nodejs.org/en/docs

8. Express.js Documentation. https://expressjs.com

9. Node.js File System API. https://nodejs.org/api/fs.html

10. Render Documentation. https://render.com/docs

11. Vercel Documentation. https://vercel.com/docs

12. Postman Learning Center. https://learning.postman.com

13. Jones, M., Bradley, J. and Sakimura, N., JSON Web Token (JWT), RFC 7519, IETF.

14. OWASP Foundation, OWASP Top Ten Web Application Security Risks. https://owasp.org/www-project-top-ten